Organização do Chess Open Source

O projeto é dividido em camadas, cada uma numa pasta e cada camada só pode depender das que estão abaixo dela.

Visão geral

```
xadrez/
├── main.py
├── requirements.txt
├── README.md
├── .env                        (nunca vai para o Git)
├── .env.example
├── .gitignore
├── core/
│   ├── __init__.py
│   ├── pecas/
│   │   ├── __init__.py
│   │   ├── peca.py
│   │   ├── peao.py
│   │   ├── torre.py
│   │   ├── cavalo.py
│   │   ├── bispo.py
│   │   ├── rainha.py
│   │   └── rei.py
│   ├── casa.py
│   ├── tabuleiro.py
│   ├── validador.py
│   ├── jogador.py
│   └── jogo.py
├── database/
│   ├── __init__.py
│   ├── conexao.py
│   ├── modelos.py
│   └── repositorios/
│       ├── __init__.py
│       ├── jogador_repo.py
│       ├── partida_repo.py
│       └── fila_repo.py
├── matchmaking/
│   ├── __init__.py
│   ├── elo.py
│   ├── fila.py
│   └── pareador.py
├── servicos/
│   ├── __init__.py
│   ├── auth_servico.py
│   ├── partida_servico.py
│   └── matchmaking_servico.py
├── interface/
│   ├── __init__.py
│   ├── app.py
```

```
│   ├── telas/
│   │   ├── __init__.py
│   │   ├── login.py
│   │   ├── lobby.py
│   │   └── partida.py
│   └── componentes/
│       ├── __init__.py
│       ├── tabuleiro_widget.py
│       └── tema.py
└── tests/
    ├── test_pecas.py
    ├── test_tabuleiro.py
    ├── test_jogo.py
    ├── test_elo.py
    ├── test_pareador.py
    └── test_repositorios.py
```

A pasta `servicos/` ela faz a ligação entre a interface e o resto, para que as telas não precisem conhecer banco nem matchmaking.

Regra de dependências

A seta mostra quem pode importar quem: interface → servicos → matchmaking, database e core.

- `core` não importa nada das outras pastas. É Python puro.
- `matchmaking` contém cálculos puros e não conhece banco nem tela.
- `database` conhece SQL e o `core`, mas nunca a interface.
- `interface` só fala com `servicos`, nunca com SQL direto.

Isso permite testar o xadrez sem banco nem tela, trocar SQLite por Postgres sem tocar nas regras e trocar o customTkinter por outra interface no futuro.

Arquivos da raiz

- `main.py`: ponto de entrada. Chama `criar_tabelas()`, cria a janela `AppXadrez` e executa `mainloop()`. Não tem regra de negócio. Sempre rode da raiz: `python main.py`.
- `requirements.txt`: lista de dependências, como customtkinter, sqlalchemy, psycopg2-binary, python-dotenv, alembic, bcrypt e pytest. Instalação: `pip install -r requirements.txt`.
- `README.md`: o que é o projeto, como instalar, como configurar o `.env`, como rodar e como testar.
- `.env`: variáveis secretas, principalmente `DATABASE_URL`. Em desenvolvimento use `sqlite:///xadrez.db`; em produção, a URL do banco hospedado (por exemplo Neon).
- `.env.example`: cópia do `.env` com valores falsos. Esta vai para o Git, para mostrar quais variáveis o projeto precisa.
- `.gitignore`: deve listar `.env`, `__pycache__/`, `*.db` e `.venv/`.

core/ — as regras do xadrez

É a pasta mais importante e a mais estável. Responde: o que é uma peça, como ela anda, o que é uma jogada válida e de quem é a vez. Não usa SQLAlchemy nem customTkinter.

core/pecas/

- `peca.py`: classe abstrata `Peca` (herda de `ABC`). Guarda `cor`, `linha` e `coluna`. Tem o método abstrato `movimentos_validos(tabuleiro)`, que cada subclasse é obrigada a implementar. Também tem o método auxiliar `_movimentos_em_linha(tabuleiro, direcoes)`, usado por Torre, Bispo e Rainha, além de `mover_para`, `posicao` e `__str__`.
- `peao.py`: `Peao`. Anda 1 casa (2 no primeiro movimento) e captura na diagonal. Dependendo da cor, a direção é -1 ou +1.
- `torre.py`: `Torre`. Só define `DIRECOES` (4 retas) e chama `_movimentos_em_linha`.
- `bispo.py`: `Bispo`. Só define as 4 diagonais.
- `rainha.py`: `Rainha`. Define as 8 direções (retas e diagonais).
- `cavalo.py`: `Cavalo`. Lista de 8 `SALTOS` em L; não desliza, então pula peças.
- `rei.py`: `Rei`. 8 direções, mas só 1 casa. Roque e xeque ficam para depois.
- `__init__.py`: reexporta as classes para permitir `from core.pecas import Peao, Torre`.

Demais arquivos do core

- `casa.py`: `Casa` com `linha`, `coluna` e `peca` (ou `None`). Método `esta_vazia()`.
- `tabuleiro.py`: `Tabuleiro`, matriz 8x8 de `Casa` (composição). Métodos: `dentro_dos_limites`, `get_casa`, `get_peca`, `colocar_peca`, `remover_peca`, `mover_peca` e `_montar_posicao_inicial`. É o único lugar que mexe em `self.casas`. O `__str__` serve para depurar no terminal.
- `validador.py`: `ValidadorDeMovimento` com `validar_movimento(tabuleiro, peca, linha, coluna)`. Hoje confere se o destino está em `movimentos_validos`; no futuro entram xeque, xeque-mate e empate.
- `jogador.py`: `Jogador`, o papel na partida (nome, cor, `pecas_capturadas`, `registrar_captura`). Não é a conta do banco: a conta é `JogadorDB`, em `database/`.
- `jogo.py`: `Jogo`, a classe que amarra tudo. Possui o `Tabuleiro`, os dois `Jogador`, o `turno_atual` e o `historico_movimentos`. Método principal: `mover_peca(origem, destino)`, que valida, registra captura, move, grava no histórico e troca o turno. Cria o validador só na hora do uso (dependência).

database/ — persistência com SQLAlchemy

É a única camada que conhece SQL. Guarda contas, partidas, movimentos e fila de espera.

- `conexao.py`: lê `DATABASE_URL` do `.env`, cria o `engine` (use `pool_pre_ping=True`, para reconectar se o banco hospedado tiver dormido), a `SessionLocal` (sessionmaker), a classe `Base` e a função `criar_tabelas()`.
- `modelos.py`: classes ORM, uma por tabela. Use o sufixo `DB` para não confundir com as classes do `core`:
  - `JogadorDB`: id, nome, senha_hash, rating (começa em 1200), criado_em.
  - `PartidaDB`: id, branco_id e preto_id (chaves estrangeiras para `JogadorDB`), resultado, inicio, fim.
  - `MovimentoDB`: id, partida_id, numero, origem, destino, peca.
  - `FilaEsperaDB`: jogador_id, rating, entrou_em.
- `repositorios/jogador_repo.py`: criar jogador, buscar por nome, atualizar rating.

- `repositorios/partida_repo.py`: criar partida, salvar movimento, finalizar partida com resultado, listar partidas de um jogador.
- `repositorios/fila_repo.py`: entrar na fila, sair da fila, listar quem espera.

Regras dos repositórios: recebem uma `Session` pronta, não fazem `commit` (quem decide é o serviço) e não contêm regra de xadrez nem de matchmaking. Só leem e gravam.

matchmaking/ — quem joga contra quem

Contém só lógica pura, sem banco e sem tela, o que facilita muito os testes.

- `elo.py`: calcula o novo rating após uma partida. Função `calcular_novo_rating(rating_a, rating_b, resultado, k=32)`. A expectativa de vitória é `1 / (1 + 10 ** ((rating_b - rating_a) / 400))`, e o resultado vale 1 (vitória), 0,5 (empate) ou 0 (derrota).
- `fila.py`: `FilaDeEspera`, estrutura em memória com `entrar`, `sair` e a lista de quem espera, guardando o horário de entrada de cada jogador.
- `pareador.py`: `encontrar_par(fila)`. Compara jogadores por rating dentro de uma tolerância que aumenta com o tempo de espera (por exemplo, 100 pontos + 10 por segundo esperado). Devolve a dupla escolhida ou nada.

servicos/ — a cola entre as camadas

Cada serviço executa um caso de uso completo, abre a sessão, chama as outras camadas, faz o `commit` e devolve o resultado.

- `auth_servico.py`: cadastrar e entrar. Gera o hash da senha com bcrypt e confere no login. Nunca guarde senha em texto puro.
- `partida_servico.py`: iniciar partida (cria `PartidaDB` e o `Jogo` do core), jogar um lance (chama `Jogo.mover_peca` e, se for válido, salva o `MovimentoDB`) e encerrar (grava o resultado e atualiza os ratings usando `elo.py`).
- `matchmaking_servico.py`: entrar na fila, sair da fila e procurar adversário (usa `fila_repo` e `pareador`).

interface/ — customTkinter

Só desenha e captura cliques. Não tem regra de xadrez, nem SQL.

- `app.py`: janela principal (`AppXadrez`, herda de `ctk.CTk`). Guarda a tela atual e troca entre login, lobby e partida.
- `telas/login.py`: campos de nome e senha, botões entrar e cadastrar, chama `auth_servico`.
- `telas/lobby.py`: mostra nome e rating do jogador, botão procurar partida, estado da fila e histórico recente.
- `telas/partida.py`: o tabuleiro, de quem é a vez, peças capturadas, desistir. Chama `partida_servico`.
- `componentes/tabuleiro_widget.py`: o grid 8x8 de botões, os símbolos Unicode das peças, a seleção e o destaque dos movimentos possíveis. É o código do `app.py` atual, movido para cá.
- `componentes/tema.py`: cores e fontes (casa clara, casa escura, seleção, destaque) em um lugar só.

tests/

Um arquivo de teste por módulo, rodados com `python -m pytest` a partir da raiz.

- `test_pecas.py`, `test_tabuleiro.py`, `test_jogo.py`: testam o `core` (por exemplo, peão inicial tem 2 movimentos).
- `test_elo.py` e `test_pareador.py`: testam as funções puras do matchmaking.
- `test_repositorios.py`: usa um banco em memória (`sqlite:///:memory:`), então não depende da internet nem suja o banco real.

Fluxo de uma jogada

1. O jogador clica numa casa do `tabuleiro_widget`.
2. A tela `partida.py` chama `partida_servico.jogar(...)`.
3. O serviço chama `Jogo.mover_peca`, que consulta o `ValidadorDeMovimento`.
4. Se for válida, o `Tabuleiro` move a peça e o turno troca.
5. O serviço grava o `MovimentoDB` pelo `partida_repo` e faz o `commit`.
6. A tela redesenha o tabuleiro.

Fluxo de uma partida com matchmaking

1. O jogador entra pela tela de login.
2. No lobby, clica em procurar partida: `matchmaking_servico` coloca o jogador na fila.
3. O `pareador` encontra um adversário com rating próximo.
4. `partida_servico` cria a `PartidaDB` e o `Jogo`.
5. Os dois jogam; cada lance é salvo.
6. No fim, o `elo.py` calcula os novos ratings e o serviço os grava e finaliza a partida.

Ordem de implementação

1. Mover `models/` e `logica/` para `core/` e corrigir os imports; conferir que o jogo ainda roda.
2. Criar `conexao.py` e `modelos.py` com SQLite local.
3. Criar os repositórios e testá-los em memória.
4. Implementar `elo.py` e `pareador.py` com testes.
5. Criar os serviços.
6. Quebrar a interface atual em `app.py`, telas e componentes.
7. Trocar a `DATABASE_URL` para o banco hospedado.
8. Atualizar o diagrama de classes.

Como funcionam os imports

- Use imports absolutos a partir da raiz: `from core.jogo import Jogo`.
- Toda pasta precisa de um `__init__.py` (pode ficar vazio).
- Sempre execute a partir da pasta raiz `xadrez/`, senão o Python não acha os pacotes.

Erros comuns

- Importar `database` ou `interface` dentro de `core`: quebra a regra de dependências.

- Colocar SQL ou `commit` dentro das telas.
- Subir o `.env` para o GitHub com a senha do banco.
- Guardar senha sem hash.
- Misturar as classes ORM (`JogadorDB`) com as do domínio (`Jogador`): são coisas diferentes.
- Esquecer o `__init__.py` e receber `ModuleNotFoundError`.

Se o jogo for online

Com jogadores em computadores diferentes, o app não deve falar direto com o banco, senão a senha dele fica dentro do programa. Nesse caso nasce uma pasta `api/` (FastAPI) entre a interface e os serviços: o app chama a API por HTTP, e só a API conhece o `DATABASE_URL`. As pastas `core/`, `database/`, `matchmaking/` e `servicos/` passam a rodar no servidor, sem mudança de código.

Para o diagrama de classes

Classes novas a incluir: `JogadorDB`, `PartidaDB`, `MovimentoDB` e `FilaEsperaDB`; os repositórios; `FilaDeEspera` e `Pareador`; e os serviços. Relações principais: `PartidaDB` se associa a dois `JogadorDB` (branco e preto), `MovimentoDB` é composição de `PartidaDB`, os repositórios dependem de `Session`, e os serviços dependem de repositórios, `Pareador` e `Jogo`.